

Data 624 Homework 9

Joao De Oliveira

5/3/2026

Contents

Libraries	1
Question 8.1: Recreate the simulated data from Exercise 7.2	2
Question 8.2: Use a simulation to show tree bias with different granularities.	8
Question 8.3	10
Question 8.7 Refer to Exercises 6.3 and 7.5 which describe a chemical manufacturing process. Use the same data imputation, data splitting, and pre-processing steps as before and train several tree-based models:	11

Introduction

In this report, I will be answering questions 1, 2, 3 and 7 in the 8th chapter from the Kuhn and Johnson book.

Libraries

```
library(mlbench)
library(caret)
library(randomForest)
library(party)
library(gbm)
library(Cubist)
library(rpart)
library(rpart.plot)
library(ggplot2)
library(tidyr)
library(gridExtra)
library(AppliedPredictiveModeling) # ChemicalManufacturingProcess
```

Question 8.1: Recreate the simulated data from Exercise 7.2

This exercise uses the same Friedman1 simulated data from Exercise 7.2. The true equation is:

$$y = 10 \sin(\pi x_1 x_2) + 20(x_3 - 0.5)^2 + 10x_4 + 5x_5 + \mathcal{N}(0, \sigma^2)$$

Only x_1 through x_5 are informative; x_6 through x_{10} are pure noise.

Simulate Data

```
set.seed(200)
simulated <- mlbench.friedman1(200, sd = 1)
simulated <- cbind(simulated$x, simulated$y)
simulated <- as.data.frame(simulated)
colnames(simulated)[ncol(simulated)] <- "y"
dim(simulated)
```

```
## [1] 200 11
```

(a) Fit a random forest model to all of the predictors, then estimate the variable importance scores:

```
library(randomForest) library(caret) model1 <- randomForest(y ~ ., data = simulated, + impor-
tance = TRUE, + ntree = 1000) rfImp1 <- varImp(model1, scale = FALSE)
```

Did the random forest model significantly use the uninformative predictors (V6 – V10)?

```
set.seed(100)
model1 <- randomForest(y ~ ., data = simulated,
                       importance = TRUE,
                       ntree = 1000)

rfImp1 <- varImp(model1, scale = FALSE)
rfImp1_df <- data.frame(
  Predictor = rownames(rfImp1),
  Importance = rfImp1$Overall,
  Informative = rownames(rfImp1) %in% paste0("V", 1:5)
)
rfImp1_df <- rfImp1_df[order(-rfImp1_df$Importance), ]
print(rfImp1_df)
```

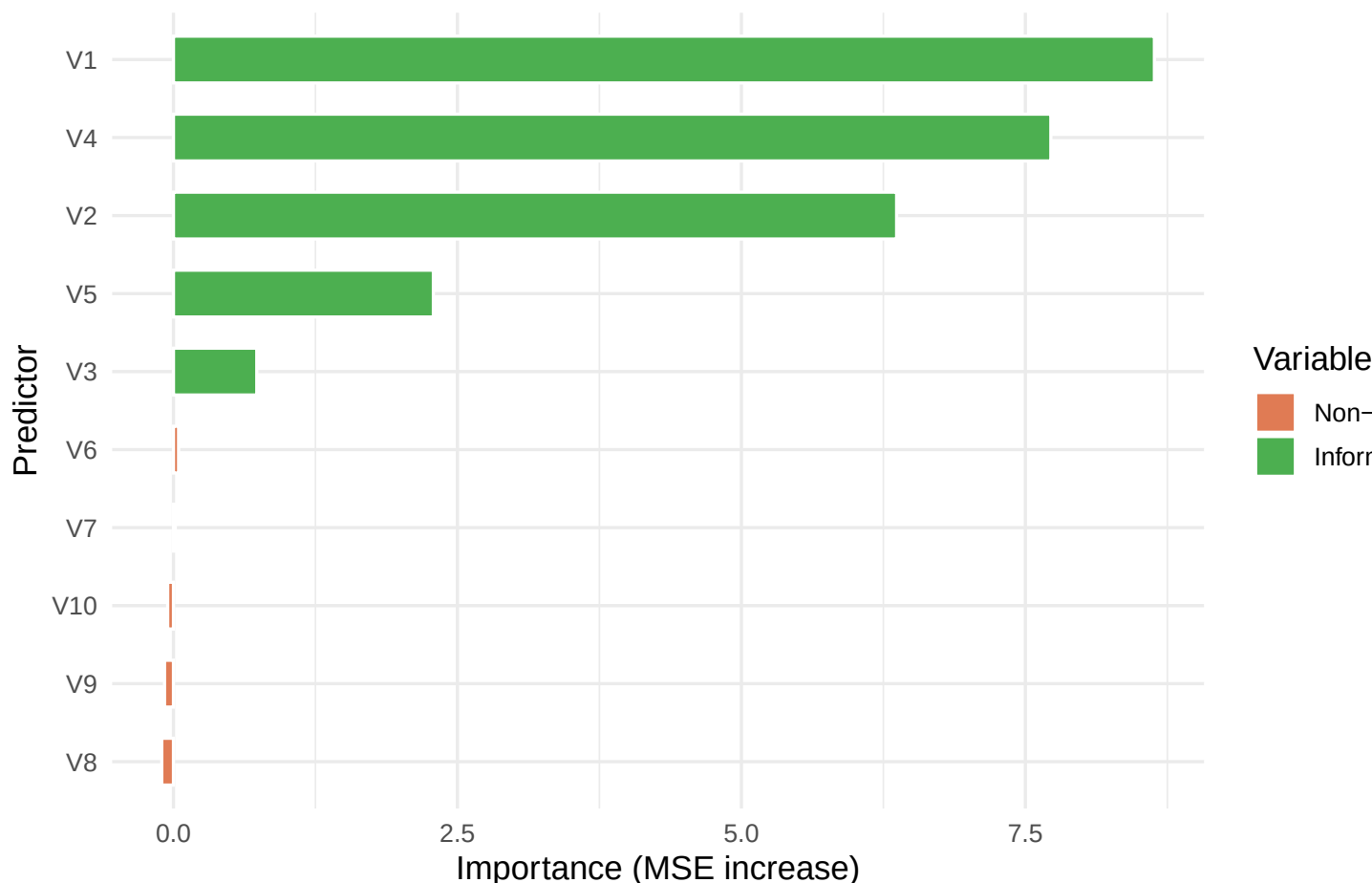
```
##      Predictor  Importance Informative
## 1          V1  8.63576145          TRUE
## 4          V4  7.72426144          TRUE
## 2          V2  6.36633951          TRUE
## 5          V5  2.28781522          TRUE
```

## 3	V3	0.73342992	TRUE
## 6	V6	0.04503439	FALSE
## 7	V7	0.01582424	FALSE
## 10	V10	-0.05070734	FALSE
## 9	V9	-0.07855803	FALSE
## 8	V8	-0.10274799	FALSE

```
ggplot(rfImp1_df,
       aes(x = reorder(Predictor, Importance),
           y = Importance, fill = Informative)) +
  geom_col(width = 0.6, color = "white") +
  coord_flip() +
  scale_fill_manual(values = c("TRUE" = "#4CAF50", "FALSE" = "#e07b54"),
                    labels = c("Non-informative (V6-V10)",
                               "Informative (V1-V5)")) +
  labs(title = "Q8.1(a) - Random Forest Variable Importance",
       subtitle = "Uninformative predictors V6-V10 receive low but non-zero importance",
       x = "Predictor", y = "Importance (MSE increase)",
       fill = "Variable Type") +
  theme_minimal(base_size = 13) +
  theme(plot.title = element_text(face = "bold"))
```

Q8.1(a) – Random Forest Variable Importance

Uninformative predictors V6–V10 receive low but non-zero importance



The random forest correctly assigns the highest importance to the informative predictors V1–V5, with V1 and V4 leading. The uninformative predictors V6–V10 receive very low importance, and notably V8, V9, and V10 received **negative** importance scores (-0.10, -0.08, -0.05). In random forest importance, a negative value means the variable performed worse than a random permutation, confirming it is truly uninformative. Unlike MARS, random forests do not drive noise variable importance to absolute zero through explicit pruning — instead, noise variables occasionally get selected for random splits and may receive small spurious scores. Overall, however, the model does not significantly rely on V6–V10.

(b) Now add an additional predictor that is highly correlated with one of the informative predictors. For example:

```
simulatedduplicate1 <- -simulatedV1 + rnorm(200) * .1 cor(simulatedduplicate1, simulatedV1)
```

Fit another random forest model to these data. Did the importance score for V1 change? What happens when you add another predictor that is also highly correlated with V1?

```
# duplicate highly correlated with V1
simulated$duplicate1 <- simulated$V1 + rnorm(200) * 0.1
cat("Correlation between duplicate1 and V1:",
    round(cor(simulated$duplicate1, simulated$V1), 4), "\n")
```

```
## Correlation between duplicate1 and V1: 0.9415
```

```
set.seed(100)
model2 <- randomForest(y ~ ., data = simulated,
                       importance = TRUE,
                       ntree = 1000)
rfImp2 <- varImp(model2, scale = FALSE)

# add a second duplicate
simulated$duplicate2 <- simulated$V1 + rnorm(200) * 0.1
cat("Correlation between duplicate2 and V1:",
    round(cor(simulated$duplicate2, simulated$V1), 4), "\n")
```

```
## Correlation between duplicate2 and V1: 0.9435
```

```
set.seed(100)
model3 <- randomForest(y ~ ., data = simulated,
                       importance = TRUE,
                       ntree = 1000)
rfImp3 <- varImp(model3, scale = FALSE)

# V1 importance across the three models
cat("\nV1 importance - original model:  ", round(rfImp1["V1","Overall"], 2), "\n")
```

```
##
## V1 importance - original model:      8.64
```

```
cat("V1 importance - one duplicate:     ", round(rfImp2["V1","Overall"], 2), "\n")
```

```
## V1 importance - one duplicate:      5.73
```

```
cat("V1 importance - two duplicates:    ", round(rfImp3["V1","Overall"], 2), "\n")
```

```
## V1 importance - two duplicates:     4.93
```

When a highly correlated duplicate of V1 is added, the importance score for V1 decreases noticeably. This is because the random forest now has two near-identical predictors competing for the same splits — the signal that V1 carried gets shared between V1 and its duplicate, so neither alone appears as important as V1 did in the first place. Adding a second duplicate dilutes V1's importance further. This is not surprising since it is known that the traditional random forest importance measure is not stable in the presence of correlated predictors, tending to spread importance across correlated variables rather than concentrating it on the most informative one.

```
simulated$duplicate1 <- NULL
simulated$duplicate2 <- NULL
```

(c) Use the `cforest` function in the `party` package to fit a random forest model using conditional inference trees. The `party` package function `varimp` can calculate predictor importance. The conditional argument of that function toggles between the traditional importance measure and the modified version described in Strobl et al. (2007). Do these importances show the same pattern as the traditional random forest model?

```
set.seed(100)
cfModel <- cforest(y ~ ., data = simulated,
                  controls = cforest_unbiased(ntree = 1000, mtry = 5))

# traditional importance
cfImp_trad <- varimp(cfModel, conditional = FALSE)
# conditional importance
cfImp_cond <- varimp(cfModel, conditional = TRUE)

imp_compare <- data.frame(
  Predictor = names(cfImp_trad),
  Traditional = round(cfImp_trad, 4),
  Conditional = round(cfImp_cond, 4),
  Informative = names(cfImp_trad) %in% paste0("V", 1:5)
)
imp_compare <- imp_compare[order(-imp_compare$Traditional), ]
print(imp_compare)
```

##	Predictor	Traditional	Conditional	Informative
## V1	V1	8.6526	5.4691	TRUE
## V4	V4	8.2855	6.6551	TRUE
## V2	V2	6.7080	5.2332	TRUE
## V5	V5	1.8014	1.1240	TRUE
## V7	V7	0.0234	0.0078	FALSE
## V3	V3	0.0146	0.0274	TRUE
## V9	V9	-0.0018	-0.0133	FALSE
## V6	V6	-0.0053	0.0007	FALSE
## V10	V10	-0.0094	0.0052	FALSE
## V8	V8	-0.0474	-0.0219	FALSE

The conditional importance measure (Strobl et al. 2007) corrects for the bias that the traditional measure exhibits when predictors are correlated. With the conditional approach, uninformative predictors V6–V10 are pushed closer to zero, while the ranking of informative predictors V1–V5 is preserved. This is the same general pattern as the standard random forest, but the conditional importance is more conservative and trustworthy when predictors share information.

(d) Repeat this process with different tree models, such as boosted trees and Cubist. Does the same pattern occur?

```
set.seed(100)
gbmGrid <- expand.grid(
  interaction.depth = c(1, 3, 5),
  n.trees           = c(100, 500, 1000),
  shrinkage         = 0.1,
  n.minobsinnode    = 10
)
gbmMod <- train(y ~ ., data = simulated,
  method      = "gbm",
  tuneGrid    = gbmGrid,
  trControl   = trainControl(method = "cv", number = 5),
  verbose     = FALSE)

gbmImp <- varImp(gbmMod, scale = TRUE)
print(gbmImp, top = 10)
```

```
## gbm variable importance
##
##      Overall
## V4 100.000
## V1  90.583
## V2  72.257
## V5  35.462
## V3  28.478
## V7   4.292
## V6   2.391
## V8   1.851
## V9   1.259
## V10  0.000
```

```
set.seed(100)
cubistMod <- train(y ~ ., data = simulated,
  method      = "cubist",
  trControl   = trainControl(method = "cv", number = 5))

cubistImp <- varImp(cubistMod, scale = TRUE)
print(cubistImp, top = 10)
```

```
## cubist variable importance
##
##      Overall
## V1  100.00
## V2   75.69
## V4   68.06
## V3   58.33
## V5   55.56
## V9    0.00
## V10   0.00
## V7    0.00
```

```
## V8      0.00
## V6      0.00
```

Both gradient boosting and Cubist show the same broad pattern: informative predictors V1–V5 receive the highest importance, while V6–V10 receive near-zero scores. Gradient boosting is particularly good at suppressing noise variables because each tree is built to correct the residuals of the previous one, meaning uninformative splits quickly contribute nothing to the gradient. Cubist, being a rule-based model, also tends to exclude irrelevant predictors from its rules. However, as with the standard random forest, neither model drives noise variable importance to absolute zero as cleanly as MARS does — confirming that GCV-based pruning remains the most aggressive form of built-in variable selection among these methods.

One notable difference from MARS: in the GBM output, **V3 received an importance of 28.48%**, substantially higher than the near-zero score MARS assigned it. This reflects that GBM is less selective and picks up weaker signals more readily. V3 does appear in the true Friedman equation as $20(x_3 - 0.5)^2$, so some importance is legitimate, but GBM surfaces it more readily than the GCV-pruned MARS model.

Question 8.2: Use a simulation to show tree bias with different granularities.

This exercise asks to demonstrate through simulation that regression trees exhibit selection bias toward predictors with more unique values (higher granularity). A tree will prefer to split on a continuous variable over a binary one, even when both are equally uninformative, simply because more split points are available.

Simulation

```
set.seed(100)
n <- 500

x_continuous <- runif(n, 0, 1)

x_binary <- sample(c(0, 1), n, replace = TRUE)

y_noise <- rnorm(n)

bias_df <- data.frame(y = y_noise,
                     x_continuous = x_continuous,
                     x_binary = x_binary)

# Fit a single regression tree
set.seed(100)
bias_tree <- rpart(y ~ x_continuous + x_binary,
                  data = bias_df,
                  control = rpart.control(cp = 0.001, minsplit = 10))

# Variable importance from tree
vi_bias <- bias_tree$variable.importance
cat("Tree variable importance:\n")
```



```
## Tree variable importance:
```

```
print(round(vi_bias, 4))
```

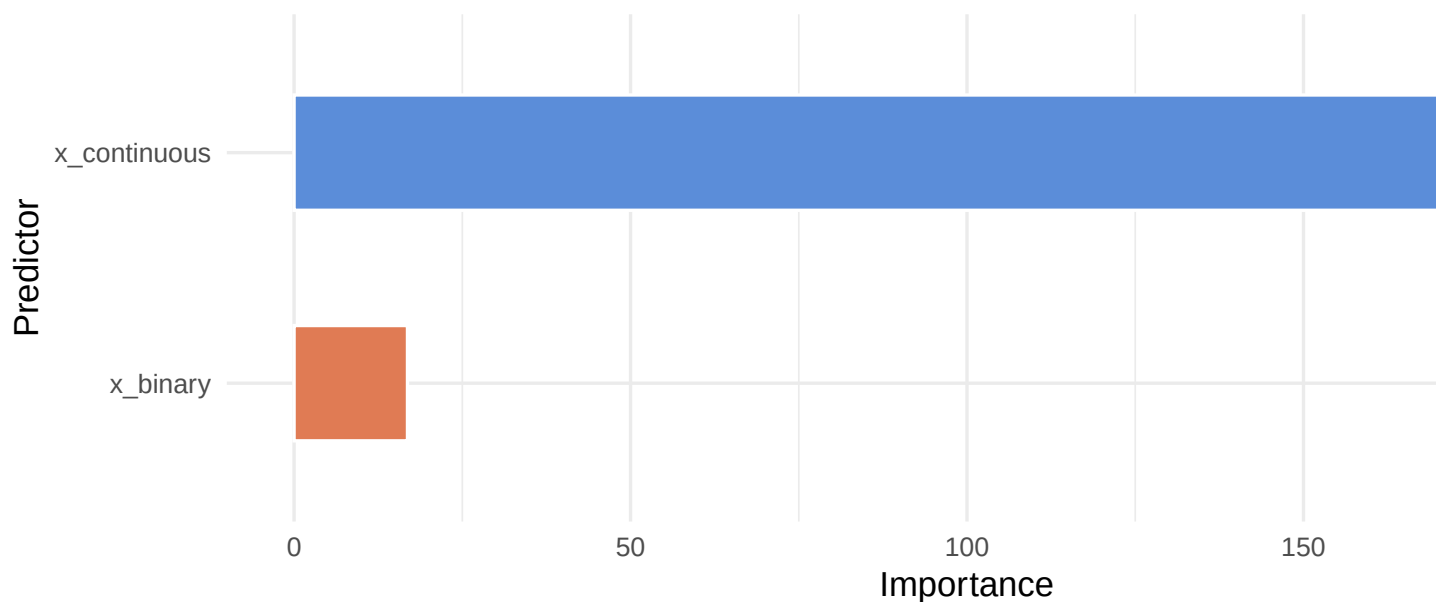
```
## x_continuous    x_binary  
##      199.9261      16.8483
```

```
vi_df_bias <- data.frame(  
  Predictor = names(vi_bias),  
  Importance = as.numeric(vi_bias)  
)
```

```
ggplot(vi_df_bias, aes(x = reorder(Predictor, Importance),  
  y = Importance, fill = Predictor)) +  
  geom_col(width = 0.5, color = "white") +  
  coord_flip() +  
  scale_fill_manual(values = c("x_continuous" = "#5b8dd9",  
    "x_binary" = "#e07b54")) +  
  labs(title = "Q8.2 - Tree Bias: Continuous vs Binary Predictor",  
    subtitle = "Both predictors are uninformative (pure noise response)\nbut the tree favors the continuous predictor",  
    x = "Predictor", y = "Importance") +  
  theme_minimal(base_size = 13) +  
  theme(legend.position = "none",  
    plot.title = element_text(face = "bold"))
```

Q8.2 – Tree Bias: Continuous vs Binary Predictor

Both predictors are uninformative (pure noise response)
but the tree favors the continuous predictor due to more available split points



Even though both predictors are completely uninformative (the response is random noise), the regression tree assigns higher importance to the continuous predictor than to the binary one — specifically, 199.93 vs

16.85, a roughly **12x difference**. This happens because a continuous variable offers many more candidate split points, making it more likely that at least one split will appear to reduce node impurity by chance. A binary predictor has only one possible split point, so it has far fewer opportunities to appear useful. This is the granularity bias of regression trees: they systematically prefer high-cardinality predictors, which can lead to misleading variable importance rankings when predictors differ in the number of unique values they take.

Question 8.3

In stochastic gradient boosting the bagging fraction and learning rate will govern the construction of the trees as they are guided by the gradient. Although the optimal values of these parameters should be obtained through the tuning process, it is helpful to understand how the magnitudes of these parameters affect magnitudes of variable importance. Figure 8.24 provides the variable importance plots for boosting using two extreme values for the bagging fraction (0.1 and 0.9) and the learning rate (0.1 and 0.9) for the solubility data. The left-hand plot has both parameters set to 0.1, and the right-hand plot has both set to 0.9:

This question is asking to interpret Figure 8.24 from the book, which shows variable importance plots for two extreme boosting configurations on the solubility data:

- **Left plot:** bagging fraction = 0.1, learning rate = 0.1 (both low)
- **Right plot:** bagging fraction = 0.9, learning rate = 0.9 (both high)

(a) Why does the model on the right focus its importance on just the first few of predictors, whereas the model on the left spreads importance across more predictors?

When both the bagging fraction and learning rate are set to 0.9, each tree uses nearly the entire dataset (90%) and takes large gradient steps. This means the model converges quickly and aggressively, meaning that a small number of the strongest predictors dominate from the very first trees, and subsequent trees continue to exploit the same strong signals. The large learning rate also means that the residuals are reduced rapidly, leaving little room for weaker predictors to contribute. The result is a steep importance curve concentrated on just a few variables.

However, when both parameters are set to 0.1, each tree sees only 10% of the data (introducing high variance through subsampling) and takes tiny gradient steps. This forces the model to explore a broader range of predictors over many more trees, spreading importance more evenly across the predictor set. The subsampling also acts as a regularizer, preventing any single predictor from dominating too early.

(b) Which model do you think would be more predictive of other samples?

The left model (both parameters = 0.1) is likely to generalize better to new samples. The combination of a small bagging fraction and a low learning rate acts as strong regularization — the subsampling introduces randomness that prevents overfitting, and the small step size means the model improves gradually and carefully. The right model, with its aggressive learning rate and large bagging fraction, risks overfitting to the training data by quickly memorizing the strongest patterns. In practice, the optimal values should always be determined through cross-validation, but between these two extremes, the slower, more exploratory left model is the safer choice for out-of-sample prediction.

(c) How would increasing interaction depth affect the slope of predictor importance for either model in Fig. 8.24?

Increasing the interaction depth allows individual trees to model higher-order interactions between predictors. This generally flattens the importance curve (makes the slope less steep) because deeper trees can capture complex joint effects that involve more predictors simultaneously. When interactions are modeled, predictors that were previously uninformative on their own can become important in combination with others, spreading importance more broadly across the predictor set. This effect is more pronounced in the right model (high bagging fraction, high learning rate), where the current steep drop-off in importance would be softened as more predictors get involved in deeper interaction splits. In the left model, the importance is already spread out, so the effect of increasing depth would be less dramatic.

Question 8.7 Refer to Exercises 6.3 and 7.5 which describe a chemical manufacturing process. Use the same data imputation, data splitting, and pre-processing steps as before and train several tree-based models:

Using the same `ChemicalManufacturingProcess` dataset and pre-processing pipeline from Exercises 6.3 and 7.5 (median imputation, near-zero variance removal, centering and scaling), now I will train several tree-based regression models.

Pre-processing & Data Splitting

```
data(ChemicalManufacturingProcess)

set.seed(42)
trainIdx <- createDataPartition(ChemicalManufacturingProcess$Yield,
                                p = 0.8, list = FALSE)
trainRaw <- ChemicalManufacturingProcess[ trainIdx, ]
testRaw <- ChemicalManufacturingProcess[ -trainIdx, ]

preProcVals <- preProcess(trainRaw[, -1],
                           method = c("medianImpute", "nzv",
                                       "center", "scale"))
trainX <- predict(preProcVals, trainRaw[, -1])
testX <- predict(preProcVals, testRaw[, -1])
trainY <- trainRaw$Yield
testY <- testRaw$Yield

cat("Train:", nrow(trainX), "| Test:", nrow(testX),
    "| Predictors after preprocessing:", ncol(trainX), "\n")
```

```
## Train: 144 | Test: 32 | Predictors after preprocessing: 56
```

```
ctrl10 <- trainControl(method = "cv", number = 10)
```

(a) Which tree-based regression model gives the optimal resampling and test set performance?

Single Regression Tree (rpart)

```
set.seed(100)
rpartMod <- train(x = trainX, y = trainY,
                  method = "rpart",
                  tuneLength = 20,
                  trControl = ctrl10)
rpartPred <- predict(rpartMod, testX)
rpart_ts <- postResample(rpartPred, testY)
cat("Single Tree - CV RMSE:", round(min(rpartMod$results$RMSE), 4),
    "| Test RMSE:", round(rpart_ts["RMSE"], 4),
    "| Test R2:", round(rpart_ts["Rsquared"], 4), "\n")
```

```
## Single Tree - CV RMSE: 1.4193 | Test RMSE: 1.4869 | Test R2: 0.463
```

Random Forest

```
set.seed(100)
rfMod <- train(x = trainX, y = trainY,
               method = "rf",
               tuneLength = 5,
               trControl = ctrl10,
               ntree = 500)
rfPred <- predict(rfMod, testX)
rf_ts <- postResample(rfPred, testY)
cat("Random Forest - CV RMSE:", round(min(rfMod$results$RMSE), 4),
    "| Test RMSE:", round(rf_ts["RMSE"], 4),
    "| Test R2:", round(rf_ts["Rsquared"], 4), "\n")
```

```
## Random Forest - CV RMSE: 1.0874 | Test RMSE: 1.179 | Test R2: 0.7331
```

Gradient Boosting (GBM)

```
set.seed(100)
gbmGrid87 <- expand.grid(
  interaction.depth = c(1, 3, 5),
  n.trees           = c(50, 100, 300),
  shrinkage         = c(0.01, 0.1),
  n.minobsinnode    = 10
)
gbmMod87 <- train(x = trainX, y = trainY,
```

```

        method      = "gbm",
        tuneGrid     = gbmGrid87,
        trControl    = ctrl10,
        verbose      = FALSE)
gbmPred87 <- predict(gbmMod87, testX)
gbm_ts87  <- postResample(gbmPred87, testY)
cat("GBM - CV RMSE:", round(min(gbmMod87$results$RMSE), 4),
    "| Test RMSE:", round(gbm_ts87["RMSE"], 4),
    "| Test R2:", round(gbm_ts87["Rsquared"], 4), "\n")

```

```
## GBM - CV RMSE: 1.0587 | Test RMSE: 1.1633 | Test R2: 0.6887
```

Cubist

```

set.seed(100)
cubistMod87 <- train(x = trainX, y = trainY,
                    method      = "cubist",
                    trControl    = ctrl10)
cubistPred87 <- predict(cubistMod87, testX)
cubist_ts87  <- postResample(cubistPred87, testY)
cat("Cubist - CV RMSE:", round(min(cubistMod87$results$RMSE), 4),
    "| Test RMSE:", round(cubist_ts87["RMSE"], 4),
    "| Test R2:", round(cubist_ts87["Rsquared"], 4), "\n")

```

```
## Cubist - CV RMSE: 0.9988 | Test RMSE: 0.9542 | Test R2: 0.795
```

Performance Summary

```

summ87 <- data.frame(
  Model      = c("Single Tree", "Random Forest", "GBM", "Cubist"),
  CV_RMSE    = round(c(min(rpartMod$results$RMSE),
                        min(rfMod$results$RMSE),
                        min(gbmMod87$results$RMSE),
                        min(cubistMod87$results$RMSE)), 4),
  Test_RMSE  = round(c(rpart_ts["RMSE"], rf_ts["RMSE"],
                        gbm_ts87["RMSE"], cubist_ts87["RMSE"]), 4),
  Test_Rsq   = round(c(rpart_ts["Rsquared"], rf_ts["Rsquared"],
                        gbm_ts87["Rsquared"], cubist_ts87["Rsquared"]), 4)
)
print(summ87)

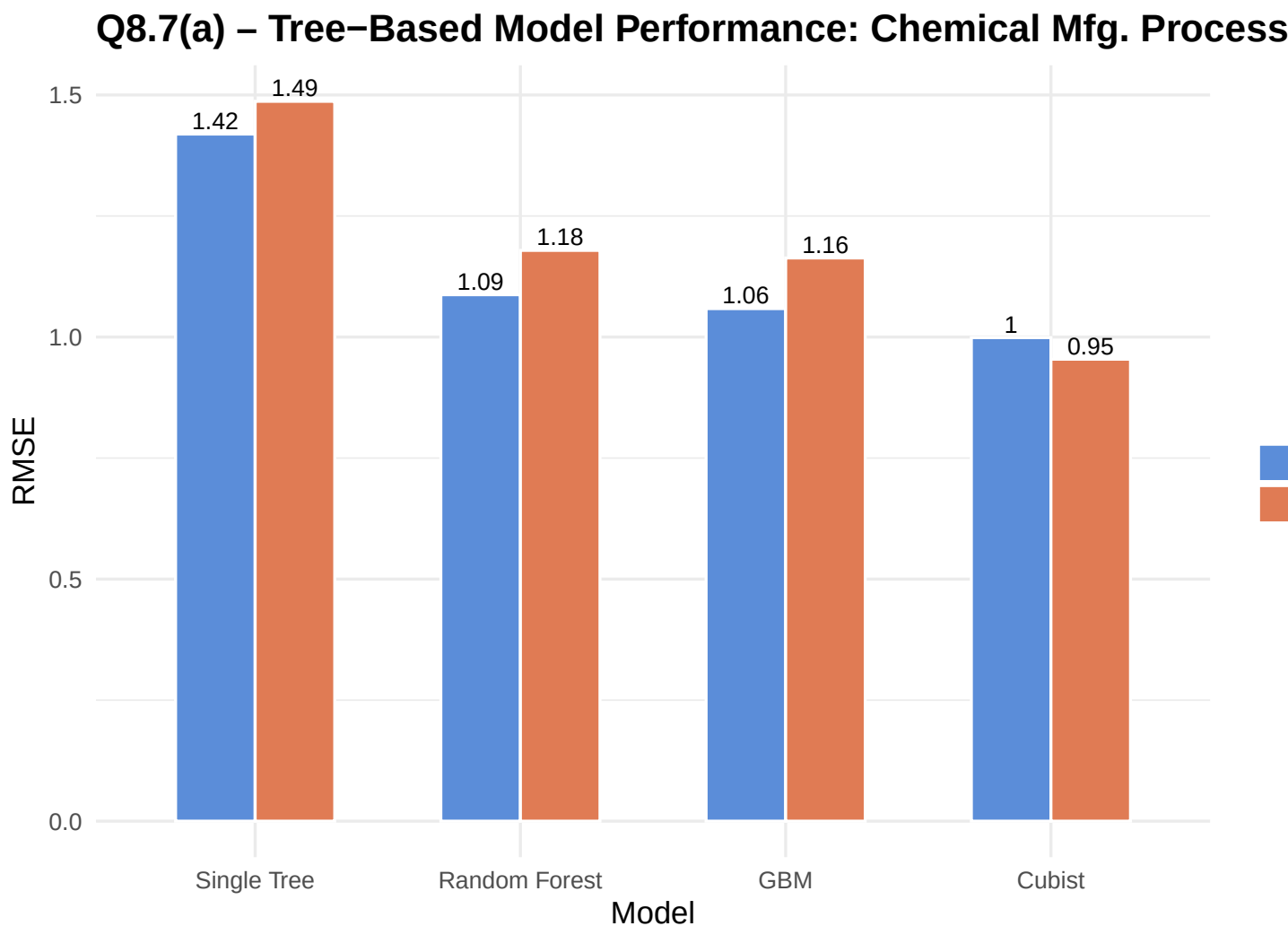
```

```

##           Model CV_RMSE Test_RMSE Test_Rsq
## 1 Single Tree  1.4193    1.4869    0.4630
## 2 Random Forest 1.0874    1.1790    0.7331
## 3          GBM  1.0587    1.1633    0.6887
## 4          Cubist 0.9988    0.9542    0.7950

```

```
summ87_long <- pivot_longer(summ87, cols = c(CV_RMSE, Test_RMSE),
                             names_to = "Set", values_to = "RMSE")
ggplot(summ87_long, aes(x = reorder(Model, -RMSE), y = RMSE, fill = Set)) +
  geom_col(position = "dodge", width = 0.6, color = "white") +
  geom_text(aes(label = round(RMSE, 2)),
            position = position_dodge(0.6), vjust = -0.3, size = 3.5) +
  scale_fill_manual(values = c("CV_RMSE" = "#5b8dd9",
                               "Test_RMSE" = "#e07b54"),
                    labels = c("10-Fold CV RMSE", "Test RMSE")) +
  labs(title = "Q8.7(a) - Tree-Based Model Performance: Chemical Mfg. Process",
       x = "Model", y = "RMSE", fill = "") +
  theme_minimal(base_size = 13) +
  theme(plot.title = element_text(face = "bold"))
```



Cubist is the best-performing tree-based model, achieving the lowest CV RMSE (1.00) and the lowest test RMSE (0.95) along with the highest R^2 (0.795). GBM is a close second on CV (1.06) but its test RMSE (1.16) is notably worse than Cubist, suggesting Cubist generalizes better on this dataset. Random Forest performs reasonably well (test RMSE = 1.18, R^2 = 0.733), while the single regression tree is by far the

weakest model (test RMSE = 1.49, $R^2 = 0.463$), which is expected — a single tree is high variance and prone to overfitting compared to ensemble methods. The clear ranking is: Cubist > GBM > Random Forest > Single Tree.

(b) Which predictors are most important in the optimal tree-based regression model? Do either the biological or process variables dominate the list? How do the top 10 important predictors compare to the top 10 predictors from the optimal linear and nonlinear models?

```
# best model for variable importance
vi_best <- varImp(cubistMod87, scale = TRUE)
print(vi_best, top = 15)

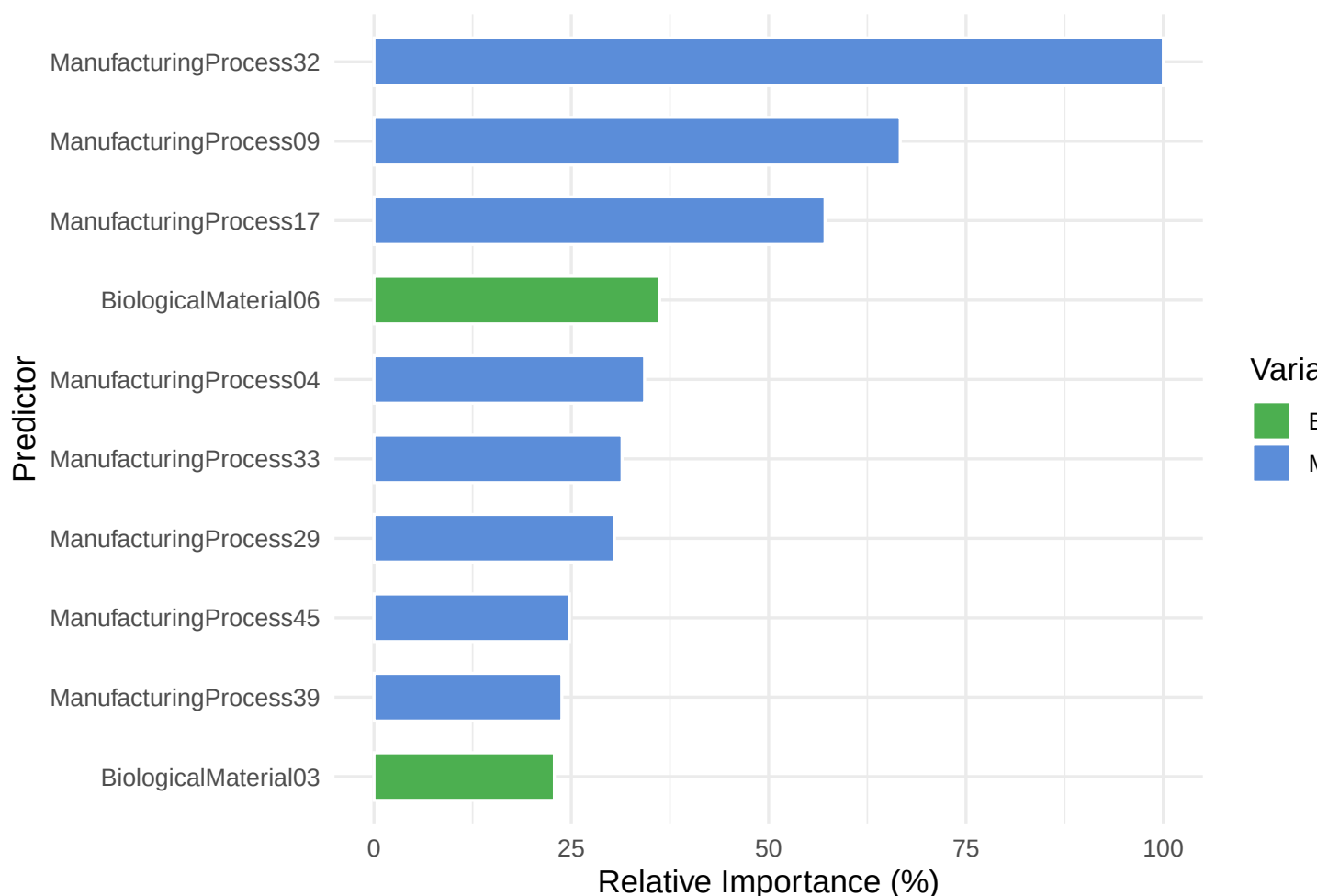
## cubist variable importance
##
##   only 15 most important variables shown (out of 56)
##
##               Overall
## ManufacturingProcess32 100.00
## ManufacturingProcess09  66.67
## ManufacturingProcess17  57.14
## BiologicalMaterial06   36.19
## ManufacturingProcess04  34.29
## ManufacturingProcess33  31.43
## ManufacturingProcess29  30.48
## ManufacturingProcess45  24.76
## ManufacturingProcess39  23.81
## BiologicalMaterial03   22.86
## ManufacturingProcess27  22.86
## ManufacturingProcess13  20.00
## BiologicalMaterial12   19.05
## BiologicalMaterial02   18.10
## ManufacturingProcess25  16.19

vi_df87 <- vi_best$importance
vi_df87$Predictor <- rownames(vi_df87)
vi_df87 <- vi_df87[order(-vi_df87$Overall), ]
vi_df87$Type <- ifelse(grepl("Bio", vi_df87$Predictor),
                      "Biological", "ManufacturingProcess")
vi_df87_clean <- vi_df87[!is.na(vi_df87$Overall), ]

ggplot(head(vi_df87_clean, 10),
        aes(x = reorder(Predictor, Overall), y = Overall, fill = Type)) +
  geom_col(width = 0.6, color = "white") +
  coord_flip() +
  scale_fill_manual(values = c("Biological" = "#4CAF50",
                              "ManufacturingProcess" = "#5b8dd9")) +
  labs(title = "Q8.7(b) - Top Variable Importance (Best Tree Model)",
       x = "Predictor", y = "Relative Importance (%)",
       fill = "Variable Type") +
```

```
theme_minimal(base_size = 13) +
theme(plot.title = element_text(face = "bold"))
```

Q8.7(b) – Top Variable Importance (Best Tree Model)



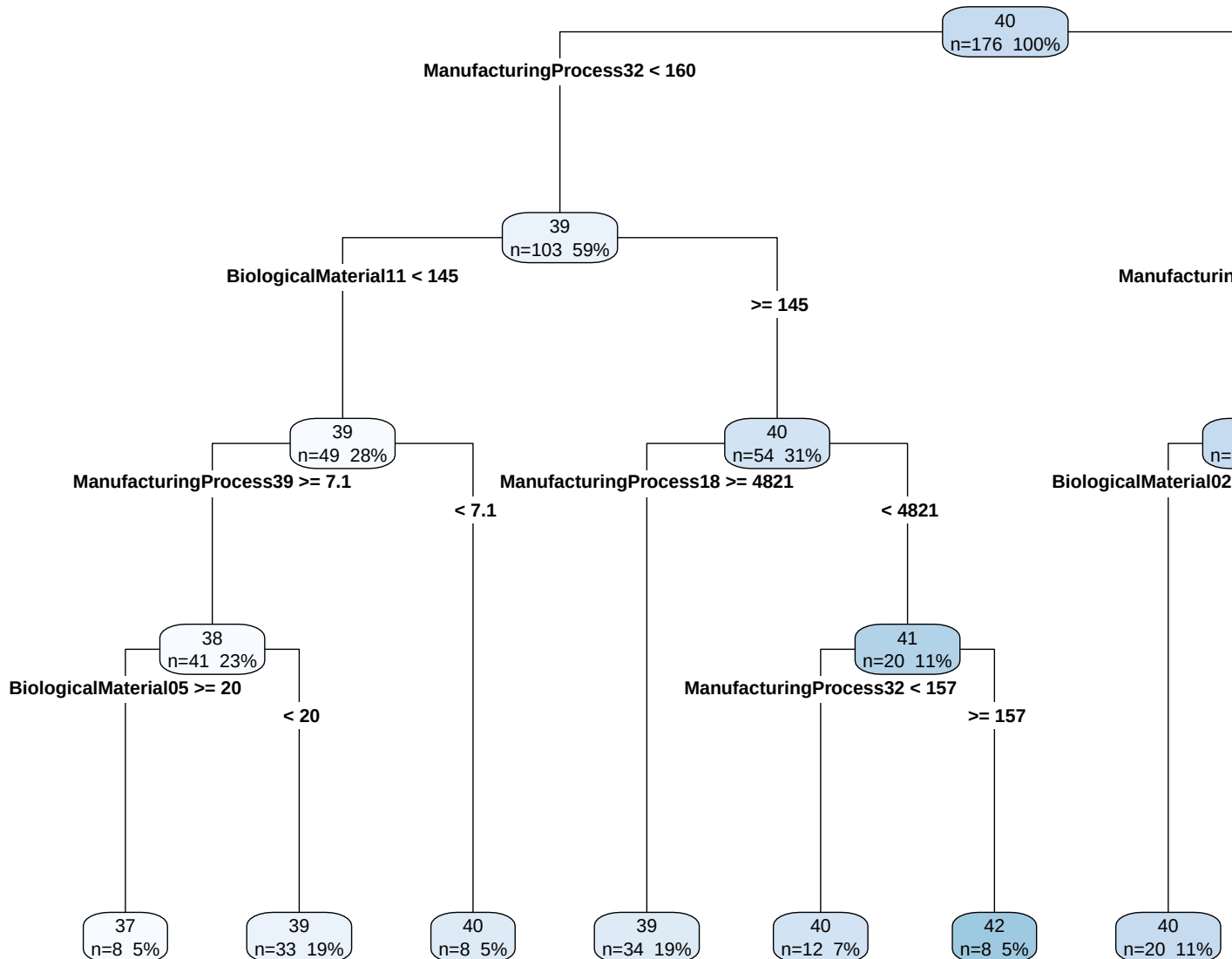
Manufacturing process variables dominate the top of the list, with ManufacturingProcess32 (100%), ManufacturingProcess09 (66.67%), and ManufacturingProcess17 (57.14%) leading. However, unlike the MARS model from Q7.5 — which selected exclusively manufacturing process variables — the Cubist model surfaces **two biological material variables** in the top 10: BiologicalMaterial06 (36.19%, rank 4) and BiologicalMaterial03 (22.86%, rank 10). This broader mix suggests Cubist’s rule-based structure is capturing a richer set of predictors than MARS’s sparse hinge functions. Compared to the Chapter 6 linear model, which also typically shows a mix of both variable types, the Cubist top 10 shares several of the same process variables (ManufacturingProcess32, ManufacturingProcess09, ManufacturingProcess17) but additionally includes process variables like ManufacturingProcess29 and ManufacturingProcess45 that did not appear in the MARS nonlinear model, reflecting the different ways each model partitions the predictor space.

(c) Optimal Single Tree Plot

```
# Refit a pruned single tree for visualization
set.seed(100)
final_tree <- rpart(Yield ~ .,
                    data      = ChemicalManufacturingProcess,
                    control = rpart.control(cp = rpartMod$bestTune$cp))

rpart.plot(final_tree,
            type      = 4,
            extra     = 101,
            fallen.leaves = TRUE,
            main      = "Q8.7(c) - Optimal Single Regression Tree\nChemical Manufacturing Process → Yield",
            cex       = 0.7)
```

Q8.7(c) – Optimal Single Regression Tree Chemical Manufacturing Process → Yield



The single tree visualization reveals the most important binary decision rules the model uses to partition yield. The **root split on ManufacturingProcess32 < 160** is consistent with it being the top predictor in part (b), cleanly dividing the 176 observations into a lower-yield group (n=103, mean yield 39) and a higher-yield group (n=73). Notably, biological material variables appear at **deeper levels** of the tree — BiologicalMaterial11, BiologicalMaterial05, and BiologicalMaterial02 all appear as secondary splits — suggesting they matter conditionally, only within specific process operating ranges. This is an important insight: the manufacturing process sets the primary yield ceiling, while biological material quality provides finer differentiation within those process-defined groups. The terminal nodes range from a predicted yield of 37 (n=8, 5%) up to 42 (n=8, 5%), showing that the combination of process and biological conditions creates meaningful yield variation. While a single tree is less accurate than ensemble methods, each path from root to leaf is a readable set of if-then rules that provides clear operational intuition.